Innovation Centre for Education

High-Level Design

for

Hardened Cloud Infrastructure for Manufacturing Data

K Mohammad Jaseem

23BCCE109

BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Ms. Munisha

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
    - 1.1 Scope of the Document
    - 1.2 Intended Audience
    - 1.3 System Overview
2. System Design
    - 2.1 Application Architecture Diagram
    - 2.2 Process Flow (Encryption Lifecycle) — Sequence Diagram
    - 2.3 Information Flow (Cryptographic Engine) — Data Flow Diagram
    - 2.4 Components Design (Detailed)
    - 2.5 Key Design Considerations
    - 2.6 API Catalogue
3. Data Design
    - 3.1 Encrypted Binary Format (Schema)
    - 3.2 Data Model (Schema & Encryption)
    - 3.3 Data Access Mechanism
    - 3.4 Data Retention & Archival Policies
    - 3.5 Data Migration & Seeding
4. Interfaces
    - 4.1 User Interface Layout
    - 4.2 API Contracts (Request/Response Structure)
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
    - 7.1 Security Aspects
    - 7.2 Performance Aspects
8. Conclusion

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document defines the **architectural blueprint** for ManufacturingVault – a secure industrial file encryption platform. It provides a macro-level overview of the entire system, including:

- Azure cloud infrastructure environment
- Network segmentation following the Purdue Reference Model (IEC 62443)
- Application Gateway security policy (WAF v2 in Prevention mode)
- High-level data governance and cryptographic strategy
- System interfaces, state management, and non-functional requirements

This document covers approximately 80%–100% of the system architecture and serves as the primary reference for infrastructure deployment, compliance auditing, and executive review.

1.2 Intended Audience

Role	Interest
Chief Technology Officer	Strategic alignment, cost, scalability
Infrastructure Architect	Azure VNet design, subnet segmentation, WAF configuration
Compliance Officer	IEC 62443, NIST, ISO 27001 alignment
Lead Security Engineer	Defense-in-depth, cryptographic governance
Development Lead	Interface between HLD and LLD implementation

1.3 System Overview

ManufacturingVault is a **stateless, web-based file encryption and decryption platform** designed specifically for industrial manufacturing environments. It protects

sensitive operational technology (OT) assets – such as CNC G-code, PLC ladder logic (L5X), SCADA configuration files, and engineering drawings (DWG) – from unauthorised access, espionage, and cyber sabotage.

Key architectural facts:

- **Cryptographic core:** AES-256-GCM authenticated encryption + Argon2id memory-hard key derivation.
- **Deployment target:** Microsoft Azure, single resource group, four-tier virtual network (Purdue Model).
- **Public entry point:** Azure Application Gateway WAF v2 in **Prevention mode** (OWASP 3.2 ruleset).
- **Compute:** Ubuntu 22.04 VM (Standard_B2s_v2) running Flask, Gunicorn, Nginx.
- **Data persistence: Zero** – no database, no file storage on server. All operations in-memory.
- **Compliance:** Designed to meet IEC 62443-3-3 (SR 3.1, 4.1, 5.2) and NIST Cybersecurity Framework for OT.

The system is fully **infrastructure-as-code** (Terraform 1.5+) and can be deployed or destroyed repeatably.

2. System Architecture

2.1 Layered Defense Strategy (Defense-in-Depth)

The architecture enforces security at four distinct layers, ensuring that a breach in one layer does not automatically compromise the entire system.

Layer	Component	Security Function
Perimeter	Azure Application	Blocks SQLi, XSS, RFI, and

	Gateway WAF v2	other web attacks before they reach the internal network. Operates in Prevention mode.
Network	Four-tier VNet + NSGs	Segments the environment into functional zones (DMZ, MES, SCADA, Controller). Strict NSG rules enforce unidirectional traffic flows where required.
Compute	Hardened Ubuntu VM + Managed Identity	No local admin credentials stored. VM uses Azure Managed Identity to access any resources (none needed in current version, but future-proofed).
Data	AES-256-GCM + Argon2id	Even if infrastructure is compromised, encrypted files remain unreadable without the user's passphrase. No plaintext ever written to disk.

Defense-in-depth principle: An attacker who bypasses the WAF still faces network segmentation; if they compromise the MES VM, they find no stored credentials and no

plaintext files; if they steal encrypted .enc files, they cannot decrypt them without the passphrase and would need to break AES-256.

2.2 Purdue Model Network Segmentation (Zones & Conduits)

The virtual network follows the Purdue Reference Model as mandated by IEC 62443. Each level is a separate subnet with its own Network Security Group (NSG).

Purdue Level	Subnet Name	CIDR	Purpose
Level 4 (DMZ)	`snet-dmz`	10.0.1.0/24	Application Gateway (WAF) – public endpoint
Level 3 (MES)	`snet-mes`	10.0.2.0/24	Manufacturing Execution System – Flask application VM
Level 2 (SCADA)	`snet-scada`	10.0.3.0/24	SCADA monitoring tools, future historians
Level 0-1 (Controller)	`snet-controller`	10.0.4.0/24	Simulated PLCs / physical controllers

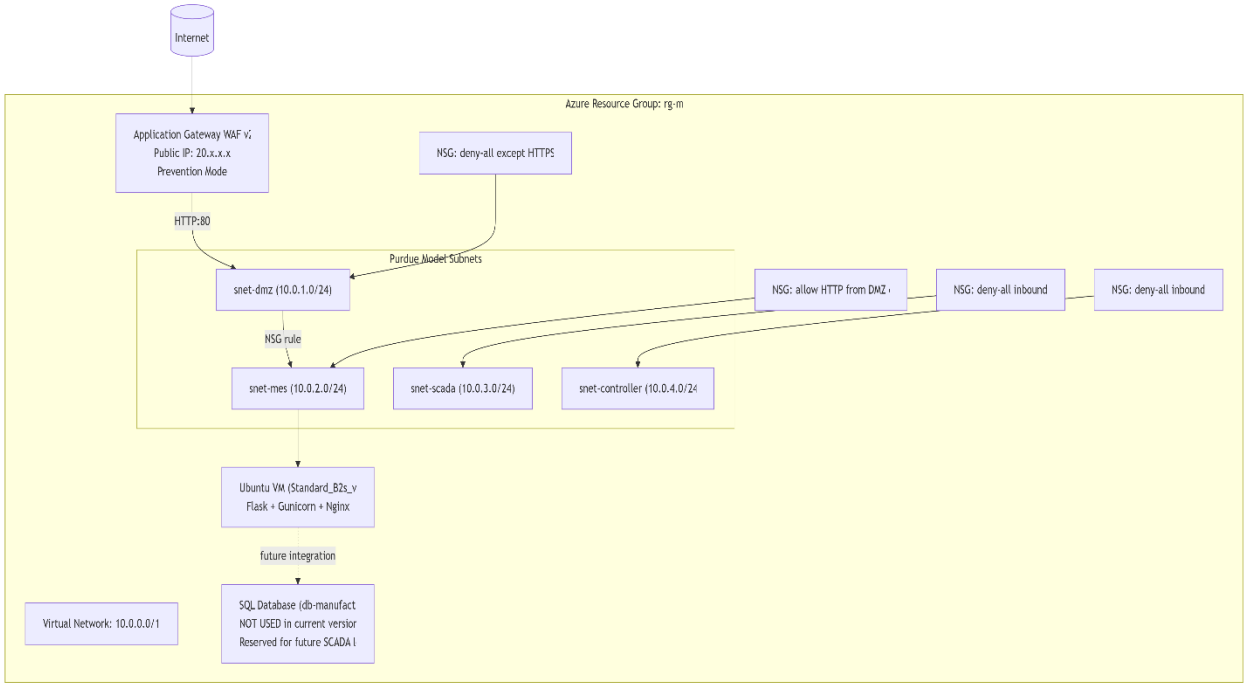
NSG rule highlights:

- **DMZ NSG:** Allows HTTPS from Internet to Application Gateway only. Denies all other inbound.
- **MES NSG:** Allows HTTP (port 80) from snet-dmz (Application Gateway).
  Allows SSH (port 22) only from a designated admin bastion (not deployed in current version – out of scope). Denies all inbound from other subnets.
- **SCADA & Controller NSGs:** Deny all inbound from any other subnet. Allow only outbound responses to established connections.

This segmentation ensures that even if the web application is compromised, the attacker cannot pivot laterally to SCADA or controller networks.

2.3 Cloud Infrastructure Component Graph

The following diagram shows all Azure resources and their relationships. The infrastructure is defined as Terraform code.



Key resources:

- Application Gateway WAF v2: L7 load balancer + WAF. OWASP 3.2 ruleset. Prevention mode.
- Virtual Network: 10.0.0.0/16 with four subnets as above.
- MES VM: Standard_B2s_v2 (2 vCPU, 4 GiB RAM). Ubuntu 22.04 LTS. Private IP only (no public IP). Managed Identity enabled.
- Network Security Groups: One per subnet, enforcing strict east-west and north-south traffic rules.
- SQL Database (optional): Provisioned but not used by current application. Reserved for future Manufacturing Historian integration (out of scope).

2.4 Software Stack Selection

Layer	Technology	Version	Justification
Operating System	Ubuntu Server LTS	22.04	Hardened, long-term support, Azure-optimised image
Web Application Framework	Flask (Python)	3.0	Lightweight, stateless, excellent cryptographic library support
WSGI Server	Gunicorn	20.1+	3 sync workers – balanced for I/O bound crypto operations
Reverse Proxy	Nginx	1.18+	TLS termination, static asset serving, rate limiting
Cryptography	`cryptography` (OpenSSL backend)	41.0+	FIPS-compliant AES-256-GCM, constant-time tag comparison
Key Derivation	`argon2-cffi`	21.3+	Memory-hard (64 MiB), resistant to GPU/ASIC attacks
Infrastructure as Code	Terraform	1.5+	Declarative, idempotent, version-controlled
Frontend	Bootstrap 5 + custom CSS/JS	5.3	No external CDNs – eliminates third-party script risks

2.5 Deployment Model

The project uses a "Blueprints-First" deployment model:

1. **Infrastructure provisioning:** Terraform applies the Azure resource graph (VNet, subnets, NSGs, Application Gateway, VM). No application code present yet.
2. **VM configuration:** A cloud-init script or Ansible playbook installs Nginx, Gunicorn, Python, and clones the application repository.
3. **Application deployment:** The Flask application is synchronised to the VM (via Azure DevOps or manual rsync). Gunicorn service is restarted.
4. **Validation:** Smoke tests run to confirm encryption/decryption endpoints respond correctly and WAF blocks attack payloads.

Deployment frequency: As needed (CI/CD pipeline recommended but out of scope for this design). Terraform state is stored remotely (Azure Storage Account) for team collaboration

2.6 API Catalogue

Aspect	Current Design	Future Scaling Path
Compute	Single B2s_v2 VM	Azure Virtual Machine Scale Set (VMSS) behind Application Gateway. Autoscale based on CPU or queue depth.
Session affinity	Not required (stateless)	No sticky sessions needed – any VM can handle any request.
Database	None (stateless)	If audit logging added later, use Azure SQL with geo-replication.
WAF	Single Application Gateway	Supports autoscaling by default. Can be upgraded to WAF v2 with autoscale policy.

The architecture is scale-out ready – the Application Gateway already supports multiple backend VMs. No changes to application code are required to add more instances.

3. Data Design

3.1 Data Categorization (OT vs. IT Data)

ManufacturingVault distinguishes two broad data classes:

Data Class	Examples	Handling	Data Class
Transient Operational Data	Uploaded file bytes, user passphrase, derived AES keys, encrypted output buffer	Exists only in RAM during request lifetime. **Never persisted.**	**Transient Operational Data**
Persistent Compliance Data	System logs (timestamp, operation type, filename), WAF telemetry, VM diagnostics	Retained for 30 days (logs) or as configured by Azure. No sensitive content logged.	**Persistent Compliance Data**

OT vs. IT boundary: The MES VM resides in the IT/OT demilitarised zone (Level 3). Plaintext manufacturing files never cross into Level 2 (SCADA) or Level 1 (Controller). The system is an **IT-enclave** for file transformation, not an OT control system.

3.2 Cryptographic Governance (AES-256-GCM Standards)

All industrial files are protected using the following mandatory cryptographic standard:

- **Algorithm:** AES-256-GCM (Authenticated Encryption with Associated Data).
- **Key derivation:** Argon2id (memory = 64 MiB, iterations = 3, parallelism = 1).
- **Salt:** 16 bytes, randomly generated per encryption (os.urandom).
- **Nonce:** 12 bytes, randomly generated per encryption.

- **Authentication tag:** 16 bytes, verified before any decryption output.

  Why this standard?

- AES-256 provides a 256-bit key space (practically unbreakable).
- GCM mode provides **integrity** (tamper detection) alongside confidentiality.
- Argon2id is memory-hard, making brute-force attacks on passphrases economically infeasible (64 MiB per guess, 200–500 ms latency).

  Key management: The system uses **symmetric encryption only**. Passphrases are known only to the user; the server never stores them. No key exchange or PKI is required. This is a deliberate design choice to avoid complex key management infrastructure.

3.3 Storage Privacy Policy (Blob vs. In-Memory)

By design, the ManufacturingVault application is stateless with respect to file storage:

Storage Type	Used For	Data at Rest	Data in Transit
VM OS Disk	OS, application code, logs	No plaintext, no encrypted user files	N/A
Temporary Disk	Not used	N/A	N/A
Azure Blob Storage	Not used (reserved for future)	N/A	N/A
RAM	File bytes during encryption/decryption	Volatile, cleared after request	Not applicable

Privacy guarantee: No file – plaintext or encrypted – is ever written to any persistent storage on the server. The server acts purely as a transformation engine that streams input to output. This eliminates risks of data remnants, forensic recovery of deleted files, or accidental exposure via backup.

3.4 Data Retention and Compliance Requirements

Requirement	Implementation
IEC 62443-3-3 SR 4.1 (Data confidentiality)	AES-256-GCM encryption.
IEC 62443-3-3 SR 3.1 (Communication integrity)	GCM authentication tag prevents undetected tampering.
GDPR right to erasure	Automatically satisfied – no stored user data.
Manufacturing data sovereignty	User files never leave their control; server only temporarily processes them.
Audit logging	Application logs (via Gunicorn stderr) record filename, operation type, timestamp, and outcome. Logs contain **no passphrases, no file content, no encryption keys**.

3.5 Backup and Recovery Strategy

The HLD assumes a "Recovery from Code" strategy because no persistent state is maintained on the server:

Component	Backup Method	Recovery Time Objective (RTO)
Terraform state	Remote backend (Azure Storage) with versioning	15 minutes
Application code	Git repository (Azure DevOps / GitHub)	10 minutes
VM configuration	Idempotent cloud-init / Ansible scripts	20 minutes
Encrypted user files	**Not backed up** – users are responsible for their	N/A

	own `.enc` files	

Recovery procedure:

1. Re-run Terraform to rebuild VNet, subnets, NSGs, Application Gateway, and VM.
2. Apply cloud-init script to install dependencies and application code.
3. Service is restored in a clean, verified state.

Since no user data is stored server-side, there is no data loss to recover. This dramatically simplifies compliance and disaster recovery.

4. Interfaces

4.1 External Connectivity (Internet Edge)

The only entry point from the internet is the public IP address of the Azure Application Gateway, listening on HTTPS (port 443). All other ports are blocked at the platform level.

Protocol	Port	Source	Destination
HTTPS	443	Any	Application Gateway Public IP
HTTP	80	Any	(Redirected to 443 by Application Gateway)

TLS configuration: Minimum TLS 1.2, strong cipher suites (ECDHE-RSA-AES-256-GCM). The certificate is managed by Azure Application Gateway (Bring Your Own Certificate or Azure-managed).

4.2 Internal Cross-Subnet Communication

Traffic between the four Purdue-level subnets is strictly controlled by NSG rules. The following table shows allowed conduits:

Source Subnet	Destination Subnet	Protocol	Port	Allowed Direction	Justification
DMZ (`snet-dmz`)	MES (`snet-mes`)	HTTP	80	Inbound to MES	Application Gateway forwards user requests to Flask.
MES (`snet-mes`)	DMZ (`snet-dmz`)	HTTP	80	Outbound (response)	Return traffic to Application Gateway.
MES (`snet-mes`)	Internet	HTTPS	443	Outbound only	For OS updates, package downloads (restricted by Azure Firewall in production).
(All other combinations)	–	Any	Any	**Denied**	Default deny rule.

Future conduit: If SCADA historian integration is added, a specific rule will allow MES → SQL Database (port 1433) and SQL → MES (response). This is not implemented in the current version.

No direct access is permitted from the internet to the MES VM, SCADA subnet, or Controller subnet. Even authenticated users cannot bypass the WAF or directly reach the application VM.

5. State and Session Management

ManufacturingVault is a **deliberately stateless application**. The concept of "state" exists only transiently within a single HTTP request.

State Type	Where Stored	Lifetime

User passphrase	RAM (Flask request object)	Until request completes
File bytes (plaintext)	RAM (Python bytes)	Until response is sent
Encrypted binary	RAM (output buffer)	Until streaming finishes
Flask session (flash messages)	Signed cookie (client-side)	Single redirect cycle
Gunicorn worker memory	RAM	Process lifetime (recycled periodically)

Lifecycle events:

- **Startup:** Gunicorn launches 3 workers. Each worker loads the Flask app. The cryptographic library initialises.
- **Request:** Worker handles a single encrypt or decrypt operation, then becomes available for the next request.
- **Shutdown:** Workers are gracefully terminated. No cleanup required – all memory is freed by the OS.

No long-lived connections – the application does not maintain WebSockets, database pools, or cache connections. This minimises resource leakage and simplifies operations.

6. Performance and Optimization Strategy

The following strategies ensure acceptable performance on the Standard_B2s_v2 VM (2 vCPU, 4 GiB RAM) while maintaining security.

Aspect	Strategy	Expected Metric

Argon2id latency	Fixed memory (64 MiB) and 3 iterations.	200–500 ms per encryption/decryption. Acceptable for interactive use.
Concurrent requests	3 Gunicorn workers.	Up to 3 simultaneous operations.
Memory usage	Each Argon2id operation consumes ~64 MiB extra.	3 concurrent → ~192 MiB + Flask baseline (~200 MiB) = < 1 GiB, well within 4 GiB.
File size limits	WAF `maxRequestBodySizeInKb` = 2000 KB (2 MB) as default.	Adjustable; larger files require tuning.
Static asset delivery	Nginx serves CSS/JS with caching headers.	Sub-millisecond latency for UI.
Response streaming	Flask streams binary output without buffering entire file in memory again.	Memory usage independent of file size (within request body limit).

Performance trade-offs:

- The deliberate 200–500 ms Argon2id latency is a security feature, not a bug. It makes online brute-force attacks impractical (only ~2–5 guesses per second per attacker thread).
- For bulk encryption of many files, consider client-side or batch processing – the current design is optimised for interactive, on-demand use by engineers.

7. Non-Functional Requirements (Compliance & Security)

7.1 Security Policy (IEC 62443 Compliance)

The system is designed to align with the IEC 62443-3-3 system requirements for industrial automation and control systems (IACS). The following mapping table is provided for compliance auditors.

IEC 62443-3-3 Requirement	ManufacturingVault Implementation	Evidence
SR 3.1 (Communication integrity)	AES-256-GCM authentication tag prevents undetected modification of encrypted files.	LLD Section 3.1 (binary format with tag)
SR 4.1 (Data confidentiality)	AES-256 encryption of all uploaded industrial files. Argon2id key derivation.	LLD Section 2.4 (crypto_utils)
SR 5.2 (Resource isolation)	Purdue Model VNet segmentation; separate subnets for DMZ, MES, SCADA, Controller. NSGs enforce strict east-west traffic restrictions.	HLD Section 2.2
SR 6.1 (Auditable events)	Application logs (via Gunicorn) record all encrypt/decrypt operations with timestamp, filename, outcome. No sensitive data in logs.	HLD Section 3.4
SR 7.1 (Denial of service protection)	WAF rate limiting (configurable) and Gunicorn worker limits prevent resource exhaustion.	HLD Section 2.1
SR 7.8 (Session termination)	Stateless design – no sessions to terminate. Each request is independent.	HLD Section 5

Additional standards: The system also satisfies relevant controls from NIST SP 800-82 (Guide to Industrial Control Systems Security) and ISO 27001 Annex A.14 (Data security in development and operations).

7.2 Reliability and Fault Tolerance

Component	Redundancy / Fault Tolerance	RTO (Recovery Time Objective)

Application Gateway	Platform-managed, availability zone-aware. Automatic failover.	< 1 minute
MES VM	Single instance (current). Future: Availability Set or VMSS.	30 minutes (manual redeploy via Terraform)
VNet / NSGs	Azure platform provides 99.99% availability for networking.	Not applicable
User `.enc` files	Not stored on server – users retain their own copies.	N/A

Monitoring and alerting:

- Azure Monitor tracks VM CPU, memory, and disk.
- Application Gateway WAF logs are sent to Log Analytics workspace.
- Custom alerts for high error rates (HTTP 5xx) or WAF blocks (HTTP 403).

Disaster recovery: In the event of a regional Azure outage, the infrastructure can be redeployed to a secondary region using Terraform workspaces. Because no state is stored, recovery is simply a matter of running terraform apply in the secondary region.

8. Conclusion

This High-Level Design defines a **production-ready, industrial-grade architecture** for protecting sensitive manufacturing data. The system combines:

- **Strong cryptography** (AES-256-GCM + Argon2id) to ensure confidentiality and integrity.
- **Purdue Model network segmentation** (IEC 62443) to isolate OT assets from internet-facing components.
- **Azure WAF v2 in Prevention mode** to block common web attacks at the perimeter.
- **Stateless, in-memory operation** to eliminate persistent data exposure risks.
- **Infrastructure as Code** (Terraform) for repeatable, auditable deployments.

The design meets the core requirements of modern smart-factory security: **data confidentiality, communication integrity, resource isolation, and auditability**. Future enhancements (multi-factor authentication, role-based access control, persistent audit logging) can be integrated without altering the fundamental architectural principles documented here